# Gabarito — Simulador Java Pleno

Resoluções de referência das 4 questões · rota `/java-test` · gerado em 14/10/2026

**Documento restrito ao avaliador.** Não compartilhe com candidatos nem versione em repositório público. Todas as resoluções Java abaixo foram executadas no próprio motor do simulador e passaram em todos os testes ocultos.

### Resumo

| # | Questão | Correção | Testes |
|---|---|---|---|
| 1 | Calcular o saldo de uma lista de lançamentos | automática | 5 |
| 2 | Totalizar as despesas por categoria | automática | 4 |
| 3 | Ranking das maiores despesas | automática | 6 |
| 4 | Deve retornar apenas as colunas necessárias em uma consulta SQL | manual | — |

### Classes de apoio (disponíveis ao candidato)

```
public enum TipoLancamento {
    RECEITA,
    DESPESA
}

public class Lancamento {
    private final String descricao;
    private final double valor;          // sempre positivo
    private final TipoLancamento tipo;
    private final String categoria;

    public Lancamento(String descricao, double valor,
                      TipoLancamento tipo, String categoria) { ... }

    public String getDescricao() { return descricao; }
    public double getValor() { return valor; }
    public TipoLancamento getTipo() { return tipo; }
    public String getCategoria() { return categoria; }
}
```

## Questão 01 — Calcular o saldo de uma lista de lançamentos

Classe `CalculadoraSaldo` · método `calcularSaldo`

> A aplicação financeira guarda receitas e despesas como objetos Lancamento (veja "Classes de apoio"). O valor de um lançamento é sempre positivo: o que define se ele soma ou subtrai é o tipo. Se a lista recebida for null ou vazia, o saldo é 0.0.
>
> **Dado** uma lista de lançamentos com receitas e despesas
> **Quando** chamo calcularSaldo(lancamentos)
> **Então** devo receber a soma das receitas menos a soma das despesas (podendo ser negativa)

### Testes ocultos (executados nesta ordem; para no primeiro que falhar)

1. calcularSaldo(lista vazia) deve retornar 0.0
2. calcularSaldo(null) deve retornar 0.0
3. calcularSaldo deve somar as receitas
4. calcularSaldo deve subtrair as despesas das receitas
5. calcularSaldo pode retornar saldo negativo

### Resolução 1: Solução com laço for-each e switch no enum

```java
import java.util.List;

public class CalculadoraSaldo {

    public double calcularSaldo(List<Lancamento> lancamentos) {
        if (lancamentos == null || lancamentos.isEmpty()) {
            return 0.0;
        }
        double saldo = 0.0;
        for (Lancamento l : lancamentos) {
            switch (l.getTipo()) {
                case RECEITA:
                    saldo += l.getValor();
                    break;
                case DESPESA:
                    saldo -= l.getValor();
                    break;
                default:
                    throw new IllegalStateException("tipo desconhecido: " + l.getTipo());
            }
        }
        return saldo;
    }
}
```

### Resolução 2: Solução com Stream

```java
import java.util.List;

public class CalculadoraSaldo {

    public double calcularSaldo(List<Lancamento> lancamentos) {
        if (lancamentos == null) {
            return 0.0;
        }
        return lancamentos.stream()
                .mapToDouble(l -> l.getTipo() == TipoLancamento.RECEITA ? l.getValor() : -l.getValor())
                .sum();
    }
}
```

### O que avaliar

- Tratar `null` (e lista vazia) antes de percorrer: sem isso o teste "calcularSaldo(null)" falha com NullPointerException.
- O valor é sempre positivo; quem decide somar ou subtrair é o `tipo`. Comparar enum com `==` ou `switch`, nunca com String.
- Saldo negativo é resultado válido: não usar `Math.abs` nem "zerar" o saldo.

### Erros comuns

- Somar todos os valores sem olhar o tipo.
- Comparar `l.getTipo().equals("RECEITA")` (String × enum: sempre false).
- Acessar `l.valor` direto: o campo é privado, o simulador acusa "tem acesso privado".

## Questão 02 — Totalizar as despesas por categoria

Classe `RelatorioDespesas` · método `totalPorCategoria`

> O relatório mensal precisa mostrar quanto foi gasto em cada categoria. Só despesas entram no relatório (receitas são ignoradas). Se não houver despesas, retorne um mapa vazio (nunca null). A lista recebida não pode ser alterada.
>
> **Dado** uma lista de lançamentos de várias categorias
> **Quando** chamo totalPorCategoria(lancamentos)
> **Então** devo receber um Map com o total gasto em cada categoria de despesa

### Testes ocultos (executados nesta ordem; para no primeiro que falhar)

1. totalPorCategoria(lista vazia) deve retornar um mapa vazio, não null
2. totalPorCategoria deve ignorar as receitas
3. totalPorCategoria deve somar as despesas de cada categoria
4. totalPorCategoria não deve alterar a lista recebida

### Resolução 1: Solução com laço e Map.merge

```java
import java.util.HashMap;
import java.util.List;
import java.util.Map;

public class RelatorioDespesas {

    public Map<String, Double> totalPorCategoria(List<Lancamento> lancamentos) {
        Map<String, Double> totais = new HashMap<>();
        if (lancamentos == null) {
            return totais;
        }
        for (Lancamento l : lancamentos) {
            if (l.getTipo() == TipoLancamento.DESPESA) {
                totais.merge(l.getCategoria(), l.getValor(), Double::sum);
            }
        }
        return totais;
    }
}
```

### Resolução 2: Solução com Stream e Collectors.groupingBy

```java
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.stream.Collectors;

public class RelatorioDespesas {

    public Map<String, Double> totalPorCategoria(List<Lancamento> lancamentos) {
        if (lancamentos == null) {
            return new HashMap<>();
        }
        return lancamentos.stream()
                .filter(l -> l.getTipo() == TipoLancamento.DESPESA)
                .collect(Collectors.groupingBy(
                        Lancamento::getCategoria,
                        Collectors.summingDouble(Lancamento::getValor)));
    }
}
```

### O que avaliar

### Erros comuns

- Retornar um mapa vazio (`new HashMap<>()`) quando não há despesas, nunca `null`.
- Filtrar só `DESPESA`; receitas não entram no relatório.
- Acumular por categoria: `merge(..., Double::sum)`, `getOrDefault` + `put` ou `groupingBy` + `summingDouble`.
- Não alterar a lista recebida (filtrar com stream ou com `if`, não com `removeIf` na própria lista).

- `put(categoria, valor)` sem somar: a última despesa sobrescreve as anteriores.
- `lancamentos.removeIf(l -> l.getTipo() == RECEITA)`: altera a lista do chamador.
- Retornar `null` quando a lista não tem despesas.

## Questão 03 — Ranking das maiores despesas

Classe `RankingDespesas` · método `maioresDespesas`

> O dashboard exibe as n maiores despesas. Ordene da maior para a menor pelo valor; em caso de empate, pela descrição em ordem alfabética. Se n for negativo, lance IllegalArgumentException com a mensagem exata "n não pode ser negativo". Se n for maior que a quantidade de despesas, retorne todas. A lista recebida não pode ser alterada.
>
> **Dado** uma lista de lançamentos e um número n
> **Quando** chamo maioresDespesas(lancamentos, n)
> **Então** devo receber as n maiores despesas, da maior para a menor, desempatando pela descrição

### Testes ocultos (executados nesta ordem; para no primeiro que falhar)

1. maioresDespesas com n negativo deve lançar IllegalArgumentException
2. maioresDespesas deve retornar as n maiores despesas, da maior para a menor
3. maioresDespesas com valores iguais deve desempatar pela descrição, em ordem alfabética
4. maioresDespesas com n maior que a quantidade de despesas deve retornar todas
5. maioresDespesas com n igual a 0 deve retornar lista vazia
6. maioresDespesas não deve alterar a lista recebida

### Resolução 1: Solução com Stream e Comparator encadeado

```java
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;
import java.util.stream.Collectors;

public class RankingDespesas {

    public List<Lancamento> maioresDespesas(List<Lancamento> lancamentos, int n) {
        if (n < 0) {
            throw new IllegalArgumentException("n não pode ser negativo");
        }
        if (lancamentos == null) {
            return new ArrayList<>();
        }
        return lancamentos.stream()
                .filter(l -> l.getTipo() == TipoLancamento.DESPESA)
                .sorted(Comparator.comparing(Lancamento::getValor).reversed()
                        .thenComparing(Lancamento::getDescricao))
                .limit(n)
                .collect(Collectors.toList());
    }
}
```

### Resolução 2: Solução com cópia da lista, sort e subList

```java
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;

public class RankingDespesas {

    public List<Lancamento> maioresDespesas(List<Lancamento> lancamentos, int n) {
        if (n < 0) {
            throw new IllegalArgumentException("n não pode ser negativo");
        }
        List<Lancamento> despesas = new ArrayList<>();
        if (lancamentos != null) {
            for (Lancamento l : lancamentos) {
                if (l.getTipo() == TipoLancamento.DESPESA) {
                    despesas.add(l);
                }
            }
        }
        despesas.sort((a, b) -> {
            int porValor = Double.compare(b.getValor(), a.getValor());
            return porValor != 0 ? porValor : a.getDescricao().compareTo(b.getDescricao());
        });
        return new ArrayList<>(despesas.subList(0, Math.min(n, despesas.size())));
    }
}
```

### O que avaliar

- Validar `n` antes de qualquer coisa, com a mensagem exata `"n não pode ser negativo"`.
- Ordem do Comparator: `comparing(getValor).reversed().thenComparing(getDescricao)`. O `reversed()` precisa vir antes do `thenComparing`; se vier no fim, inverte também o desempate.
- `limit(n)` já trata n maior que a quantidade; com `subList` é preciso `Math.min(n, size)`.
- Ordenar uma cópia (stream ou `new ArrayList<>(...)`), nunca a lista recebida.

### Erros comuns

- `Collections.sort(lancamentos, ...)` ou `lancamentos.sort(...)` direto na lista recebida.
- `...thenComparing(getDescricao).reversed()`: desempate sai em ordem alfabética invertida.
- `subList(0, n)` sem `Math.min`: IndexOutOfBoundsException quando n é maior que a lista.
- Mensagem da exceção diferente da pedida (o teste compara o texto exato).

## Questão 04 — Deve retornar apenas as colunas necessárias em uma consulta SQL (dissertativa)

> **Dado** que existe uma tabela lancamentos no banco de dados, com as colunas ao lado
> **Então** o candidato deve escrever um comando SELECT que retorne apenas as colunas descricao e valor dessa tabela
>
> Tabela `lancamentos`: `id` INT · `descricao` VARCHAR(100) · `valor` DECIMAL(10,2) · `tipo` VARCHAR(10) ('receita' ou 'despesa') · `categoria` VARCHAR(50) · `data` DATE

### Resolução esperada

```sql
SELECT descricao, valor
FROM lancamentos;
```

### Aceitar

- Maiúsculas/minúsculas, quebras de linha e ponto e vírgula final são indiferentes.
- Colunas em outra ordem (`SELECT valor, descricao`).
- Aliases (`SELECT l.descricao, l.valor FROM lancamentos l`).

### Não aceitar

- `SELECT *`: a questão pede explicitamente só as duas colunas.
- Colunas a mais (`id`, `tipo`...) ou nome de tabela errado.
- Filtros (`WHERE`) que não foram pedidos e mudam o resultado.